

Logistic Regression

Manipal University Jaipur

February 11, 2025

Table of Contents

- 1 Logistic Regression
- 2 Why not use Linear regression for classification
- 3 Logistic Regression for Multi class Classification

Introduction

- Logistic regression is one of the most popular Machine Learning algorithms, which comes under the Supervised Learning technique. It is used for predicting the categorical dependent variable using a given set of independent variables.
- Logistic regression predicts the output of a categorical dependent variable. Therefore the outcome must be a categorical or discrete value. It can be either Yes or No, 0 or 1, true or False, etc.
- but instead of giving the exact value as 0 and 1, it gives the probabilistic values which lie between 0 and 1.

Example:

- Email is spam or not.
- Will customer will buy insurance.
- Given images of a tumor we have to predict it cancer/non cancer.

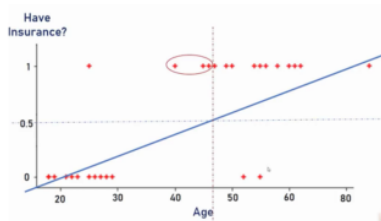
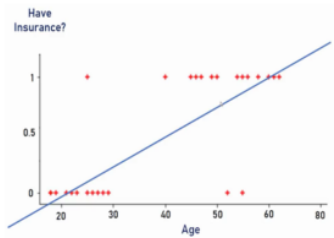
Case Study

Suppose you are working as a data scientist in a life insurance company. You are going to predict how likely a potential customer will buy your insurance product. Observing the age of customer's will know that he or she will buy an insurance product or not.

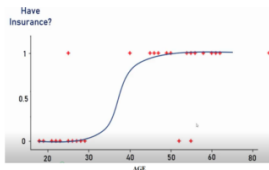
age	have_insurance
22	0
25	0
47	1
52	0
46	1
56	1
55	0
60	1
62	1
61	1
18	0
28	0
27	0
29	0
49	1

Solution

If we will going to model the problem using linear regression.



Imagine if we draw the best fit line like below. This could be much better fit as compared to the previous.



Why not use Linear regression for classification

Linear regression models the relationship as:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$$

However, in classification tasks the target variable

- y is categorical (e.g., 0 or 1).
- Linear regression can produce values outside the range $[0,1]$, making it unsuitable for probability estimation.

To address this, we use logistic regression, which maps predictions to a probability range $[0,1]$ using the sigmoid function. To ensure that the output remains between 0 and 1, we apply the sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

where z is a linear combination of input features:

Logistic Regression equation

$$Z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p \quad (1)$$

The logistic regression model equation is:

$$p(y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p)}} \quad (2)$$

- $p(y = 1|X)$ is the probability that an observation belongs to class 1.
- $1 - p(y = 1|X)$ is the probability that it belongs to class 0.
- The decision boundary is typically at 0.5:
 - If $p(y = 1|X) \geq 0.5$, predict **class 1**.
 - If $p(y = 1|X) < 0.5$, predict **class 0**.

Logistic Regression Equation

Unlike linear regression (which uses Least Squares), logistic regression uses **Maximum Likelihood Estimation (MLE)** to find the best coefficients β .

Likelihood Function

For a dataset with n observations:

$$L(\beta) = \prod_{i=1}^n p(y_i | X_i)$$

Expanding using the logistic model:

$$L(\beta) = \prod_{i=1}^n \left(\frac{1}{1 + e^{-X_i \beta}} \right)^{y_i} \left(1 - \frac{1}{1 + e^{-X_i \beta}} \right)^{(1-y_i)}$$

Log-Likelihood Function

Since products are difficult to optimize, we take the **log-likelihood**:

$$\ell(\beta) = \sum_{i=1}^n [y_i \log p(y_i | X_i) + (1 - y_i) \log(1 - p(y_i | X_i))]$$

This function is maximized to estimate the parameters β , usually using **gradient descent** or **Newton's method**.

Example

A company wants to predict whether a customer will buy a product (Yes = 1, No = 0) based on their annual income (x_1) and age (x_2)

Given:

- Intercept (β_0) = -4
- Coefficient of annual income (β_1) = 0.02
- Coefficient of age (β_2) = 0.3

Customer details:

- Annual income (x_1) = \$50,000
- Age (x_2) = 30

- 1 Compute the log-odds (z) for this customer.
- 2 Calculate the probability that the customer will buy the product.

Compute z :

$$z = -4 + (0.02 \times 50) + (0.3 \times 30)$$

$$z = -4 + 1 + 9 = 6$$

Calculate $P(y = 1 \mid x)$:

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-6}}$$

$$P(y = 1 \mid x) = \frac{1}{1 + 0.00248} = 0.9975$$

The probability that the customer will buy the product is approximately 99.75%.

Logistic Regression for Multiclass Classification

For multiclass classification:

- **One-vs-Rest (OvR):** Train multiple binary classifiers, one for each class.
- **Softmax Regression (Multinomial Logistic Regression):** Uses the **softmax function** instead of sigmoid.

$$p(y = k|X) = \frac{e^{X\beta_k}}{\sum_{j=1}^K e^{X\beta_j}}$$

where K is the number of classes.

Example

This dataset has two input variables (X_1 and X_2) and one output variable (Y). In input variables are real-valued random numbers drawn from a Gaussian distribution. The output variable has two values, making the problem a binary classification problem.

X_1	X_2	Y
2.7810836	2.550537003	0
1.465489372	2.362125076	0
3.396561688	4.400293529	0
1.38807019	1.850220317	0
3.06407232	3.005305973	0
7.627531214	2.759262235	1
5.332441248	2.088626775	1
6.922596716	1.77106367	1
8.675418651	-0.2420686549	1
7.673756466	3.508563011	1

Solution

For the given dataset, the logistic regression has three coefficients and the linear combinations of these inputs have

$$Z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$$

The job of the learning algorithm will be to discover the best values for the coefficients (β_0 , β_1 and β_2) based on the training data.

$$p(y = 1 | X) = \sigma(X\beta) = \frac{1}{1 + e^{-X\beta}}$$

To estimate the parameter β , we maximize the log-likelihood function.

$$\ell(\beta) = \sum_{i=1}^n [y_i \log \sigma(X_i \beta) + (1 - y_i) \log(1 - \sigma(X_i \beta))]$$

Since solving this equation analytically complex, we use Stochastic Gradient Descent algorithm.

Stochastic Gradient Descent algorithm

Algorithm:

- 1 **Initialize** parameters β randomly or with small values.
- 2 **For each training sample** (X_i, y_i) :
 - 1 Compute the prediction using the sigmoid function:

$$\hat{y}_i = \sigma(X_i\beta) = \frac{1}{1 + e^{-X_i\beta}}$$

- 2 Compute the gradient of the loss function (log-likelihood):

$$\nabla \ell(\beta) = X_i^T (y_i - \hat{y}_i)$$

- 3 Update the parameter β using the SGD update rule:

$$\beta = \beta + \alpha X_i^T (y_i - \hat{y}_i)$$

where α is the learning rate.

- 3 Repeat for multiple epochs until convergence.